Retrieval Augmented Generation (RAG)
Module 2 Lecture 4

Pawan Goyal
Department of Computer Science and Engineering
IIT Kharagpur

Today's Roadmap

Recap — RAG architecture · sparse vs dense retrieval

Embeddings — BERT ⟶ SBERT ⟶ modern models · choosing one

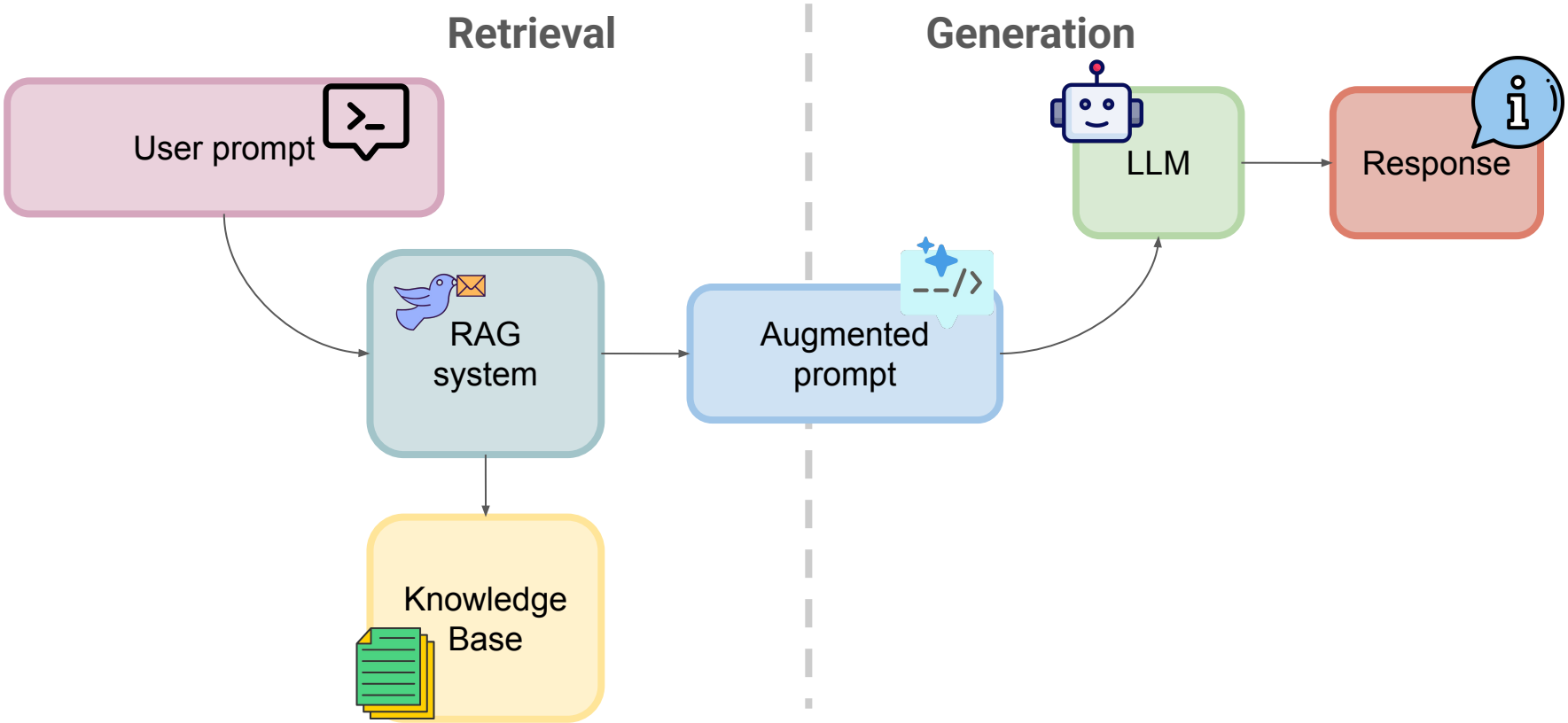
Documents — Parsing · OCR · chunking strategies

Vector store — ANN indexing: IVF · PQ · FAISS/Chroma

Hands-on — a minimal end-to-end pipeline (~40 min). We'll go step by step; run alongside us.

short live quizzes (Menti) are sprinkled through the session.

RAG architecture - *Recap From the last lecture*



RAG, fine-tuning or long context?

✓ marks a good fit. Many production systems combine RAG with fine-tuning.

If you need…	RAG	Fine-tuning	Long context
Facts that change often	✓		
Answers that cite sources	✓		✓
Search a large private corpus	✓		
A fixed tone or output format		✓	
A new skill or reasoning style		✓	
Everything fits in the prompt			✓
The lowest latency per query		✓	

Fine-tuning is covered in Module 3.

RAG or Fine-tuning? A simple decision rule

Need new KNOWLEDGE?

Facts that change, are too big to memorise, or must be traceable to a source.

→ **use RAG**

Need new BEHAVIOUR?

A fixed tone, a strict output format (e.g. always JSON), or a specialised skill.

→ **fine-tune**

Need both? Fine-tune for *how it talks*, RAG for *what it knows* — support bots, medical/legal assistants, coding assistants.

Note on Closed models

- Closed models (GPT, Claude, Gemini): you can't fully fine-tune them — at most limited API fine-tuning. RAG works with any model, open or closed.

Do GPT / Gemini already “have” RAG? And live data?

A base LLM can't see your private files or anything after its training cut-off — that gap is what RAG fills.

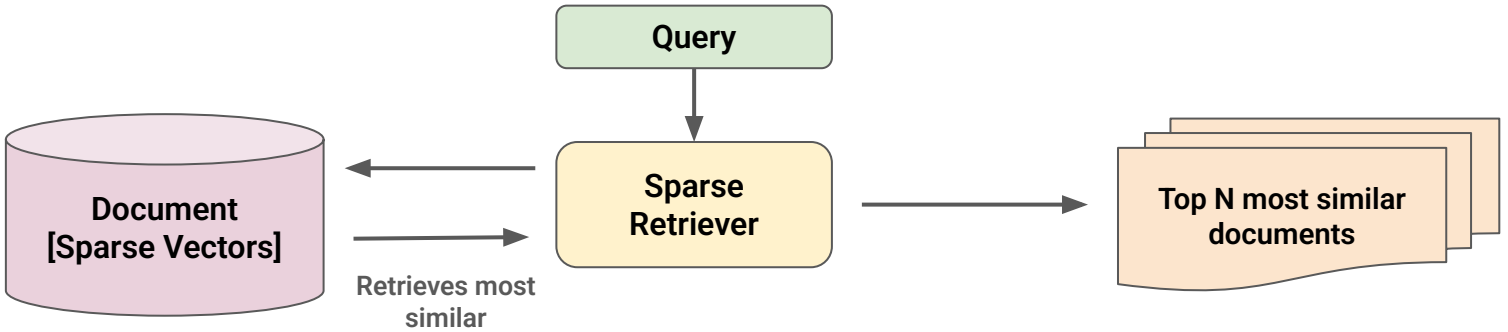
The raw model	A pre-trained LLM is frozen: it knows only its training data, has no memory of your documents, and cannot see today's news.
“ChatGPT can browse”	That web-search + read-the-page loop IS a RAG pattern — retrieve external text, then answer grounded on it. So a detailed prompt + web-search tool to research a company is a RAG system.
Your private / live data	For a data-lake, internal PDFs, or fresh records, YOU build the retrieval: index the source, fetch relevant chunks per query, feed them in. The model is unchanged — you supply the context.

Sparse Retrieval

Sparse Retrieval in RAG: Keyword-Focused Search

Method: Matches documents based on shared keywords with the user's query.

Ranking: Uses algorithms like **tf-idf** and **BM25** to score documents based on how often and how uniquely query terms appear.



Strengths: Fast and efficient, especially for large datasets.

➢ Sparse retrieval methods are often more interpretable, as they provide clear reasoning based on term occurrences, making it easier to understand why certain documents were retrieved.

Limitations of Sparse Representations - *Recap from Module 1*

🔤 No Semantic Meaning

"king" and "queen" are just two unrelated dimensions

```
"cat" ≠ "kitten" in TF-IDF
```

📏 Curse of Dimensionality

Vocabulary of 100k words = 100k-dim sparse vectors

```
99%+ entries are zero
```

🔀 No Word Order

"dog bites man" and "man bites dog" are the same

```
Bag-of-words assumption
```

🆕 Out-of-Vocabulary

New words at test time create unknown slots

```
Zero vector = no info
```

→ *These are fundamental flaws. We need a smarter representation.*

Why Sparsity Breaks Similarity

The dot product between any two one-hot vectors is always zero

Cosine Similarity between one-hot vectors:



💡 **Insight**

One-hot vectors are orthogonal by construction — the model can never discover that "cat" and "kitten" are related.
We need a representation where similar words are geometrically close.

Problem with Sparse Retrieval

Sparse retrieval matches **strings**. Dense retrieval matches **meaning**.

QUERY

"How much vacation do I get?"

DOCUMENT

"Employees accrue 18 days of paid leave annually."

BM25 / TF–IDF: zero shared words → score ≈ 0. The right answer is invisible.

Dense embeddings: "vacation" and "paid leave" sit close in vector space → high similarity.

This vocabulary-mismatch gap is the whole reason dense retrieval exists.

Dense Representations

Replace one sparse index with a short, dense vector where every dimension carries meaning

SPARSE (One-Hot)

0	0	1	0	0	0	0	0	0	0	0	0

… (50,000 dims)

✗ 99%+ zeros

✗ No similarity

✗ Huge memory

learn weights →

DENSE (Word Embedding)

0.32	-0.71	0.15	0.88	-0.43	0.07	0.61

(300 dims — every value carries meaning)

✓ All values active

✓ Geometry = meaning

✓ Compact & fast

Geometry encodes meaning — famous examples:

```
vec("king") - vec("man") + vec("woman")   ≈ vec("queen")
vec("Paris") - vec("France") + vec("Germany")   ≈ vec("Berlin")
cosine( vec("cat"), vec("kitten") )   ≈ 0.92  ✓
```

Vector Embeddings

Words

word2vec · GloVe

“bank” — river or money?

static: one vector per word, context-blind



Sentences

SBERT · E5 · GTE

“How do I open an account?”

contrastively trained: paraphrases pulled together



Chunks

long-context embedders · late chunking

a paragraph of your document

the unit of retrieval in RAG — this is what we index



Anything

CLIP · CLAP · CodeBERT

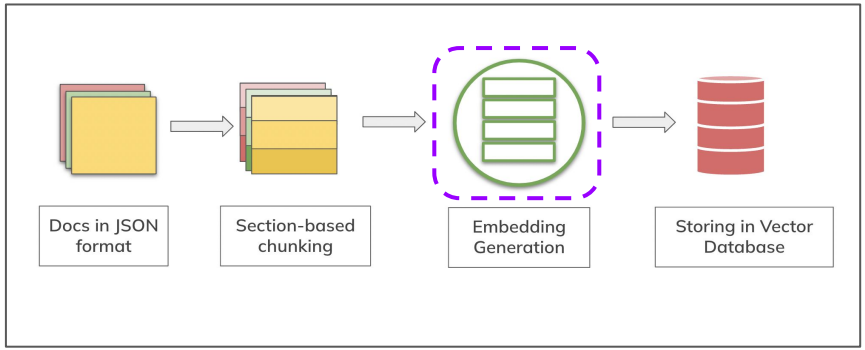
images · audio · code · tables

one shared space: a photo of a dog ≈ the word “dog”

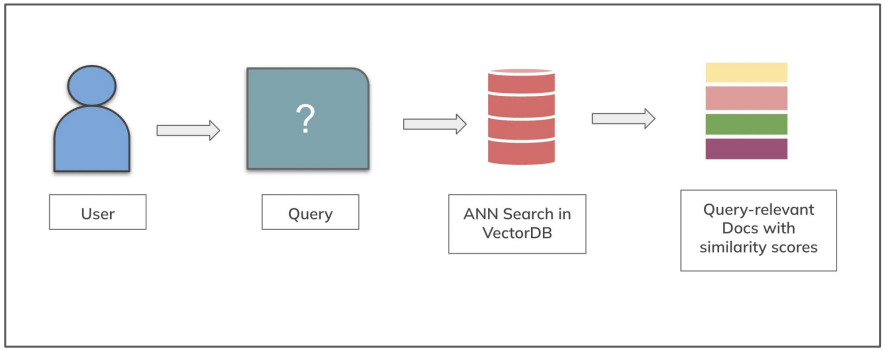


RAG with Embeddings: Pipeline

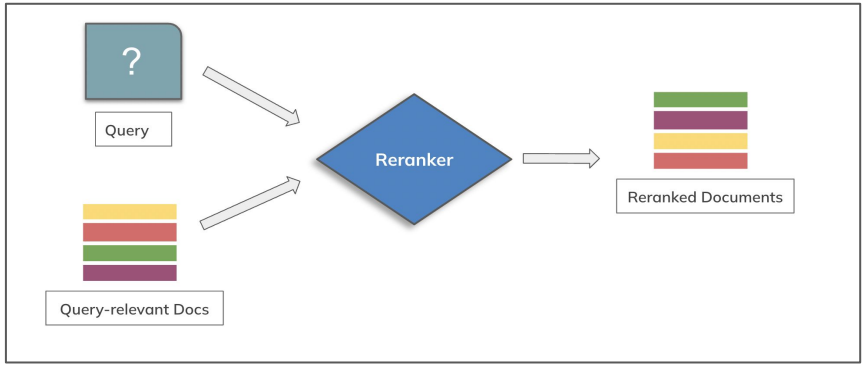
Pre-Retrieval



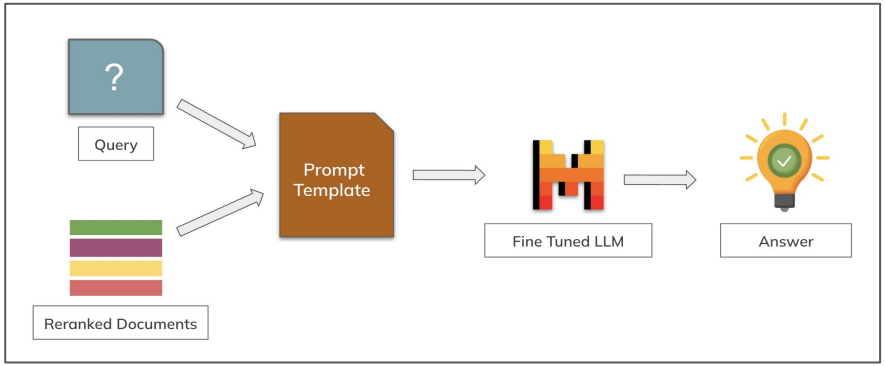
Retrieval



Post-Retrieval

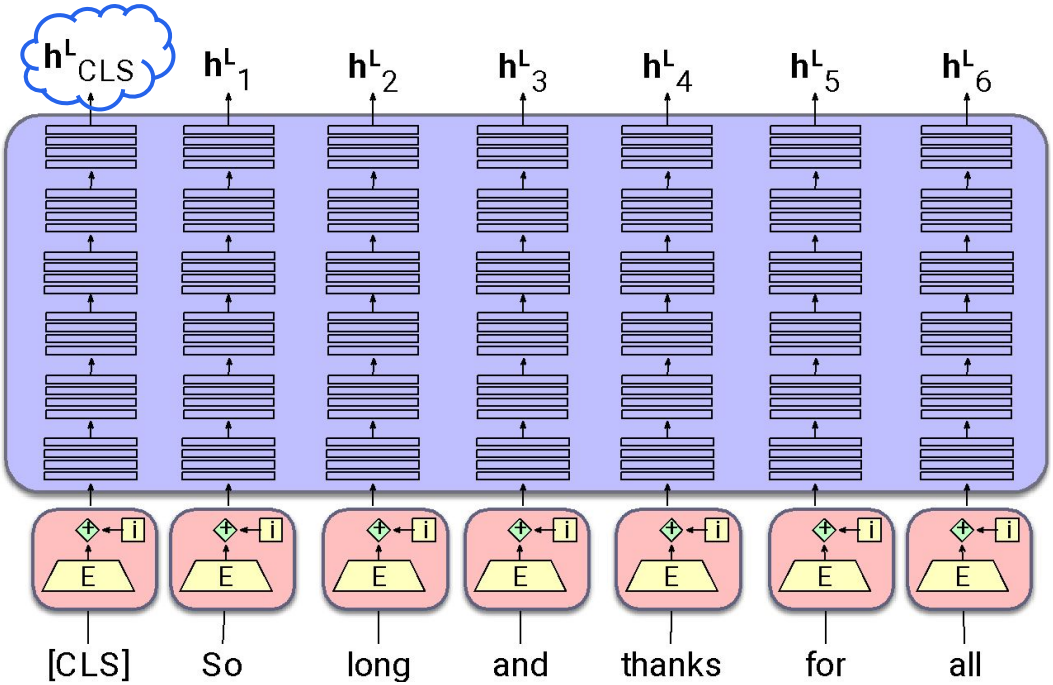


Augmentation and Generation



https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/

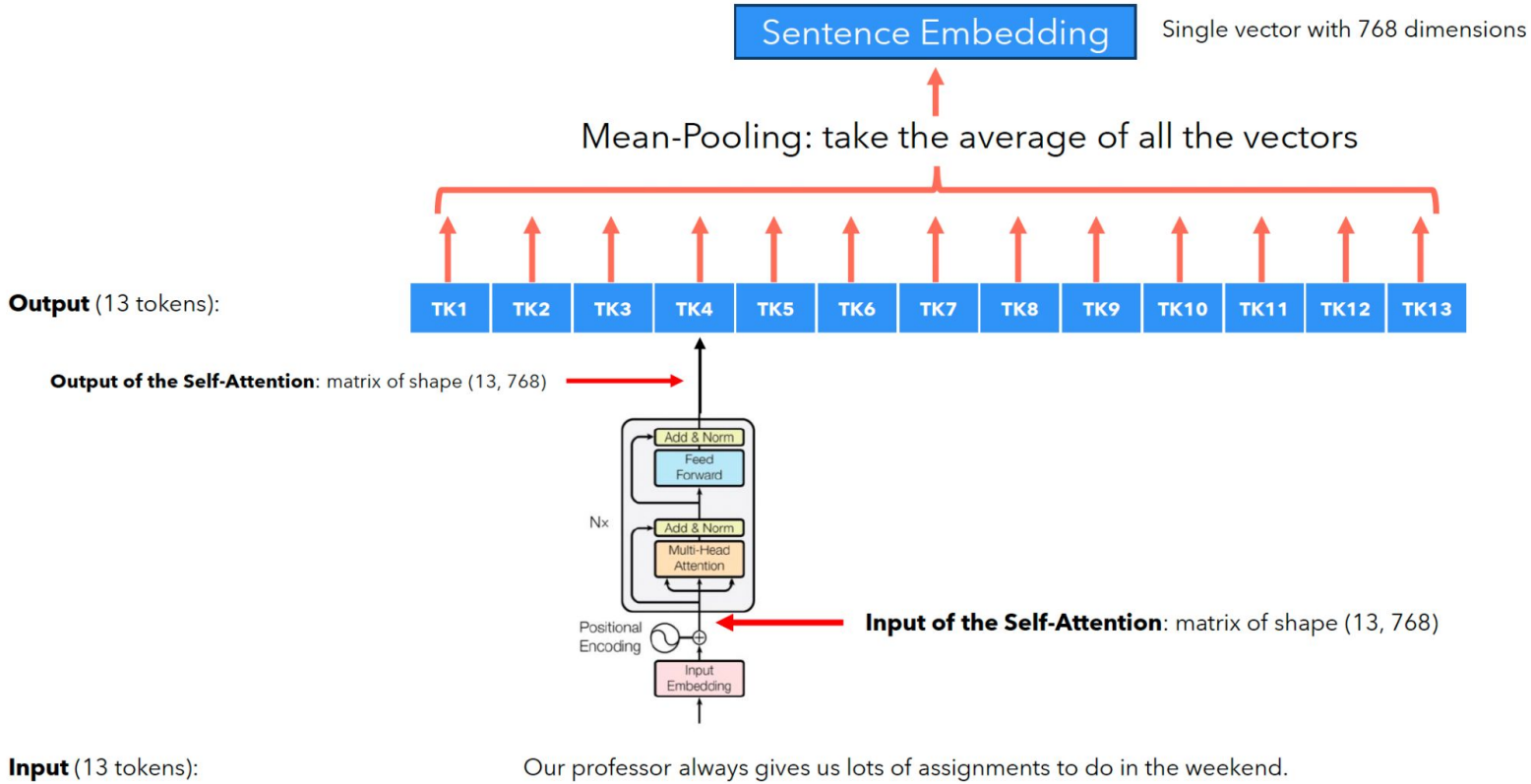
How do we get sentence embeddings using BERT?



We can get sentence embeddings from BERT using the [CLS] token, which is basically the whole sentence representation

https://web.stanford.edu/~jurafsky/slp3/

Sentence Embedding with BERT



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

Sentence Embeddings: Comparison

Comparing Sentence Embeddings

- Sentence embeddings can be compared using **cosine similarity**.
- Cosine similarity measures the angle between two embedding vectors.
- Smaller angles correspond to higher similarity scores.

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Challenge with BERT

- BERT is **not explicitly trained** to produce embeddings that are directly comparable using cosine similarity.
- Similar sentences are not guaranteed to have embeddings pointing in the same direction.
- This motivates methods for learning retrieval oriented embeddings such as **Sentence-BERT (SBERT)**, which train embeddings specifically for semantic similarity tasks.

From tokens to one vector: [CLS] vs mean-pooling

BERT gives one vector per token — you must combine them into a single sentence embedding

C [CLS] token

- A special token added at the front of the input.
- Its output vector acts as a summary of the whole sentence — like the note-taker who writes the meeting summary.
- BERT-style models have it; not every model does.

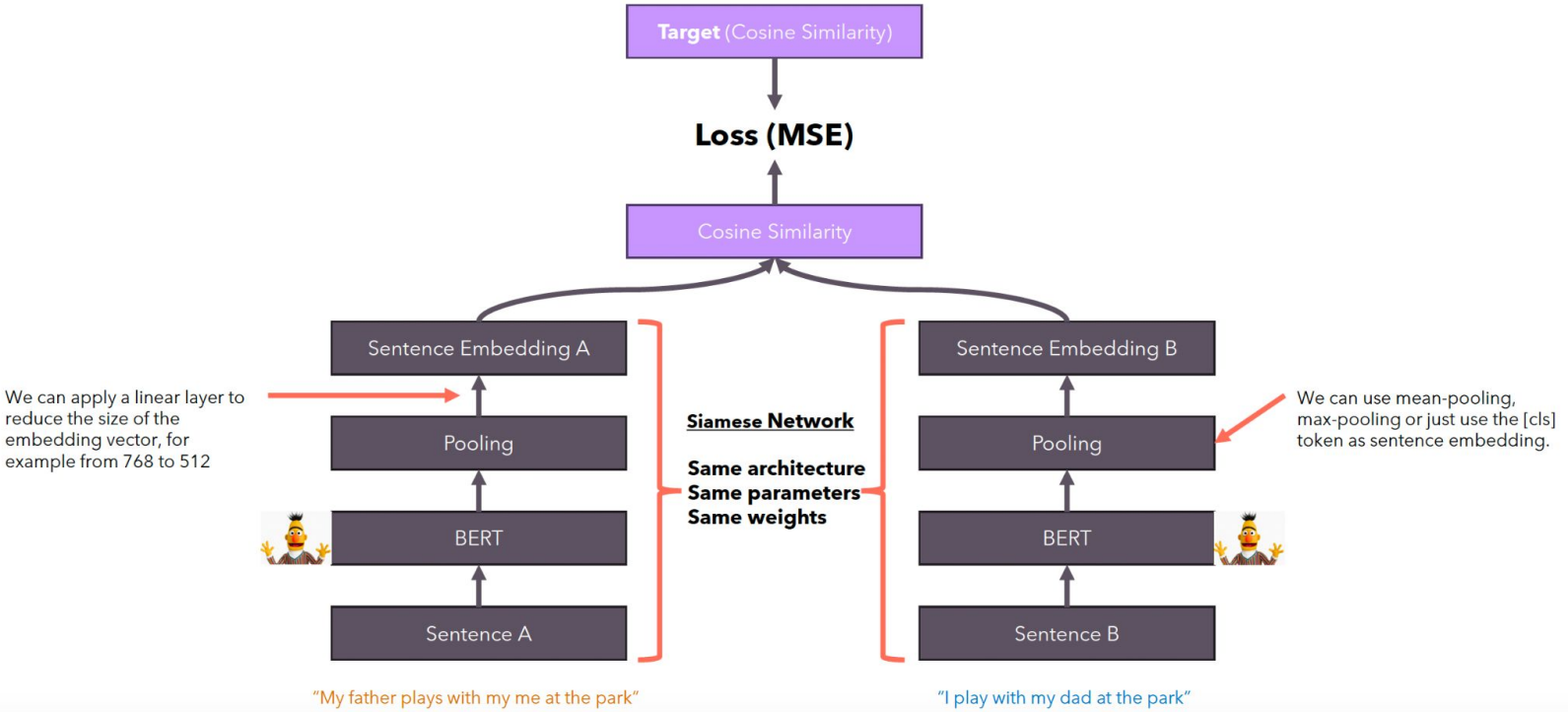
M Mean-pooling

- Average all token vectors into one.
- Model-agnostic — works even without a [CLS] token.
- SBERT's default, and usually the stronger choice for similarity.

The catch that motivates SBERT

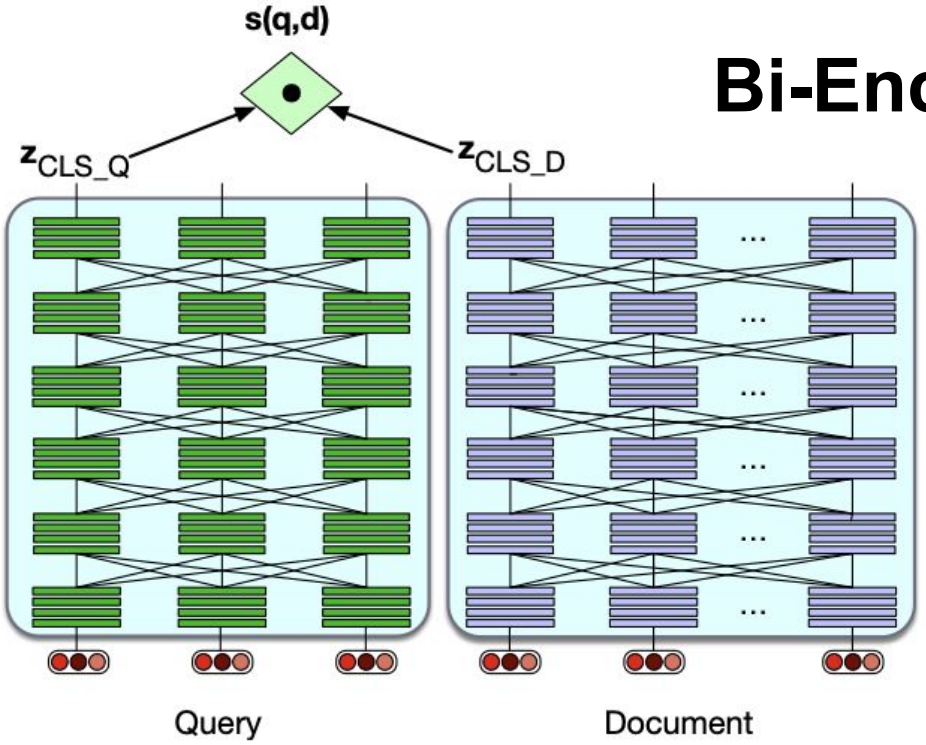
Raw BERT was never trained to make these vectors comparable by cosine similarity — two similar sentences aren't guaranteed to point the same way. SBERT fine-tunes the encoder specifically so that similar sentences land close together. That's why we use SBERT-family models for retrieval, not vanilla BERT.

Sentence BERT: Architecture [*Training Pipeline*]



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

Dense Retrieval: Finding Similarity Score

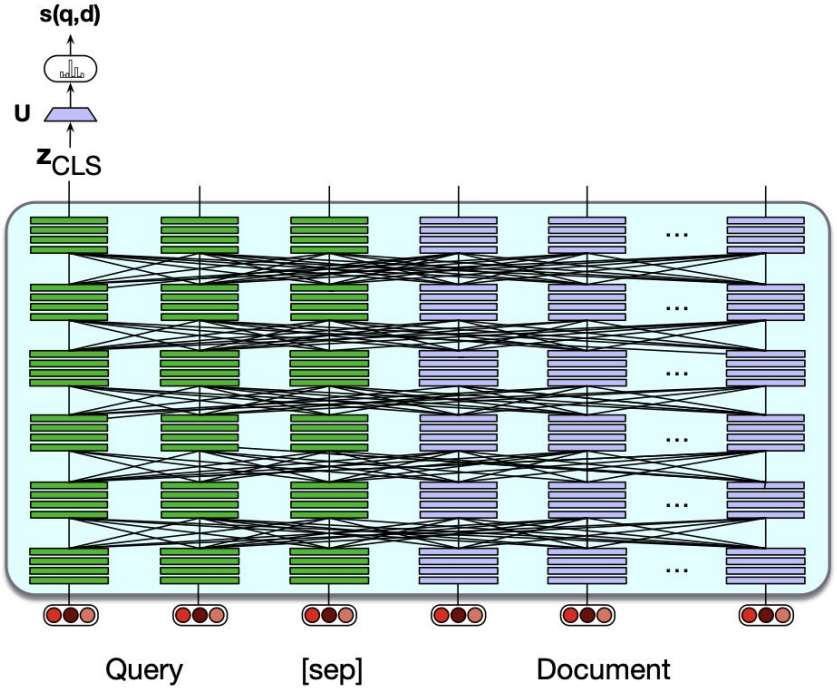


Bi-Encoder

Use separate encoders for query and document, and use the dot product between CLS token outputs for the query and document as the score. This is less compute-expensive, but not as accurate.

Dense Retrieval (costlier but more accurate)

Cross-Encoder



$$\mathbf{z} = \text{BERT}(\text{q}; [\text{SEP}]; \text{d})[\text{CLS}]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring

Generally used for Re-ranking top few

Corpus: 1,000,000 chunks. Every transformer forward pass takes 5 ms (no batching). For one query, how long until every chunk has a relevance score with a **bi-encoder**, and with a **cross-encoder**? Ignore the cost of comparing vectors.

1. Bi ≈ 83 min · Cross ≈ 83 min
2. Bi ≈ 5 ms · Cross ≈ 0.5 s
3. Bi ≈ 83 min · Cross ≈ 0.5 s
4. Bi ≈ 5 ms · Cross ≈ 83 min

Quiz 1



Bi- vs cross-encoder: cost per query

QUESTION

Corpus: 1,000,000 chunks. Every transformer forward pass takes 5 ms (no batching). For one query, how long until every chunk has a relevance score with a bi-encoder, and with a cross-encoder? Ignore the cost of comparing vectors.

ANSWER

Bi-encoder: chunk vectors are computed once, offline, so only the query is encoded → 1 pass ≈ 5 ms. Cross-encoder: every (query, chunk) pair needs its own pass → 1,000,000 × 5 ms = 5,000 s ≈ 83 min.

That is why cross-encoders only rerank a shortlist: top 100 × 5 ms = 0.5 s.

Bi-encoder vs cross-encoder in code

Bi-encoder: stored vectors, one pass per query. Cross-encoder: one pass per pair.

```python
from sentence_transformers import SentenceTransformer, CrossEncoder

docs = ["Employees accrue 18 days of paid leave annually.",
        "Unused leave up to 30 days carries over to next year.",
        "The cafeteria is open from 8 am to 6 pm."]
query = "How much vacation do I get?"

# Bi-encoder: chunks are encoded ONCE at index time; a query costs one pass
bi = SentenceTransformer("all-MiniLM-L6-v2")
doc_vecs = bi.encode(docs, normalize_embeddings=True)    # store these
q_vec = bi.encode(query, normalize_embeddings=True)
print((doc_vecs @ q_vec).round(2))                       # [0.44 0.42 0.02]

# Cross-encoder: one pass PER (query, doc) pair, so rerank only a shortlist
ce = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
print(ce.predict([(query, d) for d in docs]).round(2))   # [-5.39 -5.66 -11.25]
```

Common Embedding Models

Model	Size	Architecture
BAAI/bge-base-en-v1.5	110M	BERT
intfloat/e5-base-v2	110M	RoBERTa
nomic-ai/nomic-embed-text-v1	~500M	GPT-style
sentence-transformers/all-MiniLM-L6-v2	22M	MiniLM

For Prototyping : all-MiniLM-L6-v2

OpenAI text-embedding-3-small

text-embedding-3-small is OpenAI's small text embedding model, creating embeddings with 1536 dimensions.

Matryoshka Representation: shorten vectors via the dimensions param with graceful accuracy loss.

```python
from openai import OpenAI

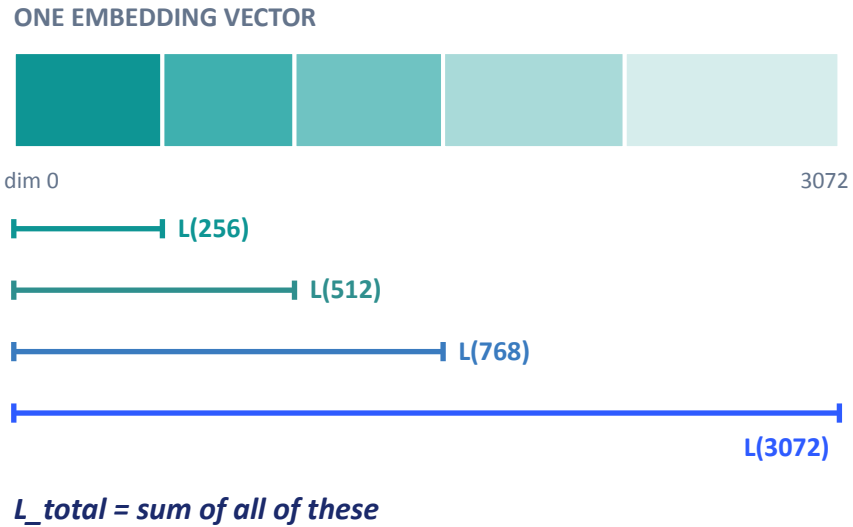
client = OpenAI()

response = client.embeddings.create(
    input="Your text string goes here",
    model="text-embedding-3-small"
)

print(response.data[0].embedding)
```

How Matryoshka embeddings order their dimensions

Same model, same forward pass. The trick is the **loss**: grade nested prefixes of the vector all at once.



Why early dims win

Dims 0–255 are scored by ***every*** term; the last dims by only one. Early dimensions feel more pressure, so the model packs the most broadly useful information there.

The analogy

Describe an animal, but you're cut off after an unknown number of words. You **front-load**: "small furry mammal…" first, fine details last. MRL hides where the cutoff is — so the model leads with what matters most.

Takeaway: the coarse-to-fine ordering is ***emergent*** - a side effect of grading every prefix. A non-MRL model never felt this pressure, so slicing its first 512 dims shreds the embedding.

Embedding dimensions: bigger is not automatically better

Dimensions are a storage–quality dial set by the model — not a "higher = better" slider, and not tied to your text length.

Two misconceptions to drop

"A 500-word para needs more dims than a 100-word one."
No — dimension count is fixed by the model. The fix for long text is chunking, not bigger vectors; a too-long chunk dilutes the vector at any size.

"Just pick the highest dimension available."
Not free — more dims mean more storage, slower search, higher cost, and shrinking (sometimes negative) accuracy gains on small data.

What actually decides it

- Storage & latency — cost scales with dims × vectors (100M × 3072-d ≈ 1.2 TB).
- Recall — more dims capture finer meaning, up to a point.
- Corpus size — small corpora gain little from huge dims.
- MRL models let you truncate dims afterwards (768 / 1536 / 3072), so the dial is cheap to test.

Rule: don't guess 768 vs 3072 — run a bake-off on YOUR data and pick the smallest dim that meets your recall target.

Embedding dimensions: a storage dial

The model fixes the dimension. More dimensions cost storage and speed, and only sometimes buy recall.

Model	Dims	Bytes per vector	100M vectors
all-MiniLM-L6-v2	384	1,536 B	≈ 154 GB
bge-base-en-v1.5	768	3,072 B	≈ 307 GB
text-embedding-3-small	1,536	6,144 B	≈ 614 GB
text-embedding-3-large	3,072	12,288 B	≈ 1.2 TB

- Long text needs chunking, not more dimensions: the model fixes the vector size.
- Bytes per vector = dims × 4 (float32): halve the dims and you halve the storage.
- Only MRL models (text-embedding-3, Nomic) can be cut to 512 or 256 dims.
- Pick the smallest model that meets your recall target on your own queries.

100M chunks embedded with text-embedding-3-large (3072 dims, float32) take ≈ 1.2 TB. You **MRL-truncate to 512 dims** and store each value as **float16** (2 bytes instead of 4). What is the new index size?

1. ≈ 205 GB
2. ≈ 154 GB
3. ≈ 102 GB
4. ≈ 51 GB



MRL + float16: how much do you save?

QUESTION

100M chunks embedded with text-embedding-3-large (3072 dims, float32) take ≈ 1.2 TB. You MRL-truncate to 512 dims and store each value as float16 (2 bytes instead of 4). What is the new index size?

ANSWER

Size = vectors × dims × bytes per dim = 100M × 512 × 2 B = 102.4 GB ≈ 102 GB.

Dims cut 3072 / 512 = 6× and precision cut 4 B / 2 B = 2×; the savings multiply: 6 × 2 = 12× smaller than 1.2 TB.

Choosing an Embedding Model

	SBERT (open-source)	OpenAI text-embedding-3
Hosting	Self-hosted / on-device	Hosted API only
Data privacy	Stays on your infra	Text sent to provider
Cost model	Compute you own (free lib)	Pay per token
Typical dims	384 – 768 (MiniLM, MPNet)	up to 3072 (resizable)
Customisation	Fine-tune on your domain	Fixed; prompt-only
Best when	Privacy, scale, custom domains	Fast start, top general quality

Design Decisions: Picking an Embedding Model

1. **The answer:** don't pick by the leaderboard - pick by **measuring candidates on your own data**. MTEB rankings are volatile and a high average score hides which model is strong on *your* task and *your* domain.

1. **The decision structure:** the shortlist itself is driven by properties of your dataset and your deployment constraints.

Picking an embedding model: start here

Your situation	Start with	Why
General English, no infra	text-embedding-3-small (API)	Cheap default, nothing to host
Data must stay in-house	BGE-M3 or Qwen3-Embedding-0.6B	Open weights run in your VPC
Hindi, Indic, mixed languages	BGE-M3 or multilingual-e5-large	Trained on 100+ languages
Code, logs, error IDs	General model + BM25 (hybrid)	Exact tokens need keyword match
Long contracts or papers	Chunk first; BGE-M3 (8K tokens)	Many models stop at 512 tokens
100M+ vectors	An MRL model with fewer dims	Storage = vectors × dims
Under ~200K tokens in total	Skip RAG: put it in the prompt	Nothing to retrieve or miss

Then compare 2–3 candidates on 50–100 of your own queries. The leaderboard only builds the shortlist.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch A - General English prose, you want zero infrastructure

Shortlist: hosted APIs - OpenAI `text-embedding-3-small/large`, Voyage `voyage-3-large`, Cohere `embed-v4`, Google `gemini-embedding`.

Why: you send text, you get vectors : no GPU, no model hosting, no version pinning. For most business RAG this is the right *first* move; the per-token cost is trivial next to an engineer's time. `text-embedding-3-small` (1536-dim) is the cheap safe default; go `-large` (3072-dim) only if the bake-off shows it earns the cost.

Branch B - Data is sensitive / must stay on-prem (privacy, compliance)

Shortlist: open-weight local - `BGE-M3`, `Qwen3-Embedding` (0.6B / 4B / 8B), `NV-Embed-v2`, `multilingual-e5-large`.

Why: healthcare, legal, finance, or internal corporate data often *legally cannot* be sent to a third-party API. Open weights run inside your VPC; the only cost is the GPU you already have. Open-source quality has largely closed the gap with commercial APIs on retrieval. **The size dial:** Qwen3-Embedding ships at 0.6B / 4B / 8B. Bigger = better recall but more VRAM, slower, costlier per embed. Start at the smallest that passes your eval - an 8B model that beats a 0.6B by 1% recall but is 5× slower is usually a bad trade for RAG.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch C - Multilingual / Indic / low-resource languages

Shortlist: `BGE-M3` (100+ languages), `multilingual-e5-large`, `Qwen3-Embedding`, NVIDIA `Llama-Embed-Nemotron-8B`, `gemini-embedding`.

Why: a model trained mostly on English will map two sentences that mean the same thing in Hindi and English to *distant* vectors - cross-lingual retrieval silently fails. Multilingual models are trained so meaning, not language, drives proximity. **The non-negotiable caveat:** "supports 100+ languages" ≠ "good at *your* language." Coverage for high-resource languages (Hindi, Arabic) is far better than for low-resource ones (e.g., Sanskrit, many African languages).

You must verify on your language with the bake-off - this is the branch where the leaderboard lies most. If retrieval is weak, fine-tuning a multilingual base on in-domain pairs often beats swapping models.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch D - Code, logs, or identifier-heavy technical text

Shortlist: code-specialized embedders (e.g. Voyage code / Jina code models) **or** a general model **plus hybrid search**.

Why: code and logs are full of exact tokens — function names, error codes, SKUs that dense semantic embeddings handle poorly . Two fixes: a model pretrained on code, *or* lean on BM25 in a hybrid setup to catch the exact tokens. Often the cheapest win is **general embedder + hybrid**, not a specialist model.
Decision rule: if your queries are mostly natural-language questions *about* code, a general model + hybrid is usually enough; if you're doing code-to-code or symbol search, a code-specialized model earns its place.

Branch E - Long documents (legal contracts, papers, transcripts)

Check first: the model's `max_seq_length`. Many embedders cap at 512 tokens and **silently truncate** longer input — you embed only the first paragraph and lose the rest.
Shortlist for long context: `BGE-M3` (8192 tokens), Jina long-context models, Nomic.
Why it's usually a chunking decision anyway: even with an 8k-token model, embedding a whole contract into one vector dilutes it. Long-context models mostly buy you *flexibility in chunk size*, not the right to skip chunking. The real choice is "chunk small + standard model" vs "chunk large + long-context model," and the bake-off decides.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch F - Huge corpus, storage/latency constrained

Shortlist: models with **Matryoshka representation learning (MRL)** - `text-embedding-3-*`, Nomic, Gemini - so you can *truncate* dimensions.

Why: storage and search cost scale with `dimensions × vectors`. At 100M vectors, 3072-dim float32 is ~1.2 TB; truncating to 512-dim cuts that ~6× for a small recall hit. MRL models are trained so the first *k* dimensions are already a usable embedding - you just slice the vector.

```python
# Matryoshka truncation: take the first N dims, then RE-NORMALIZE (critical).
import numpy as np
def truncate(vecs, dim):
    v = vecs[:, :dim]
    return v / np.linalg.norm(v, axis=1, keepdims=True)  # WHY renorm: cosine needs unit length
# Sweep dim in the bake-off: find the smallest dim that still passes your recall bar.

```

The tradeoff: dimensions are a direct storage/quality dial. Don't pay for 3072 if 768 passes your eval - but don't truncate a model that *wasn't* trained with MRL, or you'll shred the embedding.

Design Decisions Under Real Conditions: Picking an Embedding Model

Branch F - Huge corpus, storage/latency constrained

Shortlist: models with **Matryoshka representation learning (MRL)** - `text-embedding-3-*`, Nomic, Gemini — so you can *truncate* dimensions.

Why: storage and search cost scale with `dimensions × vectors`. At 100M vectors, 3072-dim float32 is ~1.2 TB; truncating to 512-dim cuts that ~6× for a small recall hit. MRL models are trained so the first *k* dimensions are already a usable embedding - you just slice the vector.

Branch G - Tiny corpus (< ~500 pages / 200k tokens)

Consider not doing RAG at all. If the whole knowledge base fits in the model's context window, just put it in the prompt - no embedder, no index, no retrieval failures. RAG is the solution for corpora too big for context; below that threshold it's complexity you don't need.

The #1 silent production RAG bug

Recall is mediocre, so the team swaps to a fancier embedding model… and it barely helps.

The real cause

A missing query: / passage: prefix on an asymmetric model (E5, BGE, GTE, Qwen, Instructor, Jina). The model treats a question like a document and collapses the asymmetry it was trained to use.

The fix (free)

Read the model card. Add the exact required prefixes. Recovers more recall than most model swaps would have gained — at zero cost.

Important point: *before reaching for a better model, verify you're using the current one correctly. The cheapest wins are configuration, not architecture.*

```python
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("intfloat/multilingual-e5-large")

# WRONG — same encoding for both. Recall drops, no warning.
# q_emb = model.encode(query); d_emb = model.encode(documents)

# RIGHT — E5/BGE-style models need these exact prefixes:
q_emb = model.encode([f"query: {query}"],     normalize_embeddings=True)
d_emb = model.encode([f"passage: {d}" for d in documents], normalize_embeddings=True)
```

Two settings that make or break search

Setting	Rule	Example
Distance metric	Use the model's training metric. Normalise vectors so dot product = cosine.	FAISS IndexFlatIP on raw vectors is not cosine: normalise first.
Dense only or hybrid	Add BM25 for exact IDs, then fuse both rankings with RRF.	“error E1047”, SKU-7731, function names

Both are settings, not model choices: check them before you swap the embedding model.

Hybrid search: dense + sparse together

Dense retrieval matches meaning; sparse (BM25) matches exact tokens. Real systems combine both so neither blind spot bites.

Dense (embeddings)

- Catches meaning: “vacation” ≈ “paid leave.”
- Misses rare exact tokens — error codes, SKUs, function names, IDs.

Sparse (BM25 / TF–IDF)

- Nails exact keywords and rare tokens.
- Blind to synonyms — zero shared words scores ≈ 0.

Combine them (hybrid)

Run both retrievers, then fuse the rankings — commonly with Reciprocal Rank Fusion (RRF), which just adds 1/(k+rank) from each list. Use it when queries mix natural language with exact identifiers (code, logs, product catalogs, legal citations). It's often a bigger win than swapping to a fancier embedding model.

Combining lists: Reciprocal Rank Fusion (RRF)

A dead-simple way to merge two ranked lists — using only the ranks

For each document, add up `1 / (k + rank)` from every list. (k ≈ 60) Higher total = better.

Dense list

1. Doc A
2. Doc B
3. Doc C

Sparse list

1. Doc C
2. Doc A
3. Doc B

Fused score (k = 60)

Doc A	`1/61 + 1/62`	0.0325	← wins
Doc C	`1/63 + 1/61`	0.0323	
Doc B	`1/62 + 1/63`	0.0320	

Doc A is high in BOTH lists → beats Doc C, which is #1 in only one.

Why it's nice: no need to make dense scores and BM25 scores comparable — you only use each doc's position. Consistent-in-both beats spiky-in-one.

Hybrid search: fuse the two rankings with RRF

Fuse ranks, not raw scores: BM25 and cosine scores sit on different scales.

```python
def rrf(rankings, k=60):
    """Reciprocal Rank Fusion: score(d) = sum over lists of 1 / (k + rank)."""
    fused = {}
    for ranking in rankings:
        for rank, doc in enumerate(ranking, start=1):
            fused[doc] = fused.get(doc, 0) + 1 / (k + rank)
    return sorted(fused.items(), key=lambda kv: kv[1], reverse=True)

# query: "error E1047 when I log in"
bm25_rank  = ["E1047-fix", "E2001-disk", "login-faq"]   # BM25: exact token
dense_rank = ["login-faq", "E1047-fix", "E2001-disk"]   # bi-encoder: meaning

for doc, score in rrf([bm25_rank, dense_rank]):
    print(f"{doc:<11} {score:.4f}")
# E1047-fix   0.0325   <- high in BOTH lists, so it wins
# login-faq   0.0323
# E2001-disk  0.0320
```

The Unifying Principle

Every embedding choice is decided by a condition.

	Choice	The condition that decides it
	**Hosted vs local embedder**	 Can the **data leave** your network? Corpus size vs API cost.
	**Which multilingual model**	 Your specific language's coverage — **measured**, not advertised.
	**Code-specialized vs general+hybrid**	 Are queries about **code**, or code-to-code?
	**Long-context model vs small chunks**	 `max_seq_length` vs your chunk strategy.
	**Embedding dimensions (MRL)**	 **Storage/latency budget** at your vector count.
	**Query/passage prefixes**	 Is the model **symmetric** or **asymmetric**? (Read the card.)
	**Distance metric**	 What the model was **trained for**.
	**Dense vs hybrid**	 Identify **density** in the corpus.
	**RAG vs just prompt**	 Does the corpus **fit in context**?

A team's RAG recall is mediocre. They swap their E5 embedding model for a fancier one and recall barely improves. What's the most likely real cause?

1. Their chunks are too large
2. They're using Euclidean distance
3. They forgot the query: / passage: prefix the asymmetric model expects

Quiz 3

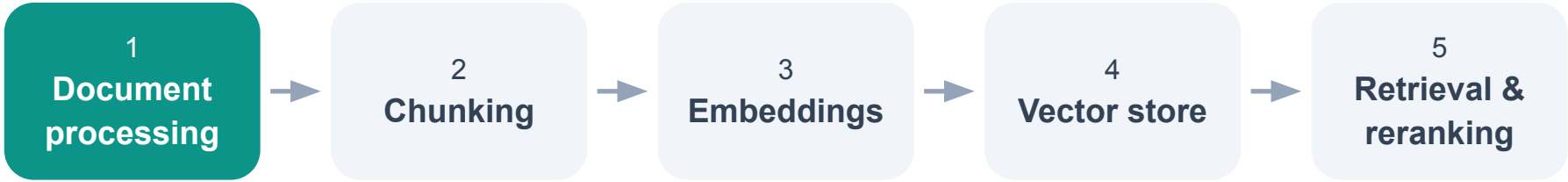


A team's RAG recall is mediocre. They swap their E5 embedding model for a fancier one and recall barely improves. What's the most likely real cause?

1. Their chunks are too large
2. They're using Euclidean distance
3. **They forgot the query: / passage: prefix the asymmetric model expects**

Design choices in a RAG system

Five decisions, made in order. Each one caps the quality of the next.



Now: Document processing

Get clean text and structure out of PDFs, scans, Word files and HTML before anything is embedded.

Document Processing Basics

Garbage in, garbage out. *Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse



Rich semantic tags (h1, p, table)
But full of boilerplate
Strip nav / headers / footers / sidebars

Document Processing Basics

***Garbage in, garbage out.** Retrieval quality is capped by how cleanly you extract text and structure before chunking.*



Built for visual fidelity, not structure
No semantic tags; multi-column text bleeds
Scanned PDFs are images → need OCR



Structured XML under the hood
Headings, lists, tables already labelled
Easiest, most reliable to parse



Rich semantic tags (h1, p, table)
But full of boilerplate
Strip nav / headers / footers / sidebars

Extraction pulls raw characters. Parsing goes further labelling what each block is (heading, table row, caption, footnote) and keeping the relationships. For RAG, that structure is as valuable as the text.

Document Processing Basics : OCR

OCR (optical character recognition) recovers text from scanned/image PDFs.

Many real-world PDFs are essentially pictures of text without OCR there's nothing to embed.

Tables are the hardest element to extract faithfully.

Best practice: emit tables as HTML to preserve row/column structure (often better than markdown).

Layout-aware models recover reading order, columns, and hierarchy, not just a flat text dump.

Layout-aware toolkits

- **Unstructured** — Element ontology + Hi-Res layout; PDF/DOCX/HTML/images
- **Docling (IBM)** — DocLayNet layout + TableFormer tables; OCR built in
- **LlamaParse** — LLM-based parsing; lightweight, good on simpler docs
- **Cloud OCR** — Azure Document Intelligence, Google Document AI

Mixed documents: route by type, don't force one pipeline

No single embedding model reads every format well. Detect the content type, then send each to the right extractor before chunking.

Text / prose

- pypdf, docx, HTML → clean text
- Standard text embedder
- The common, easy path

Tables / structured

- pdfplumber / Camelot → tables
- Serialize to Markdown/HTML, or query via SQL
- Keep rows & columns intact

Images / scans

- OCR (tesseract) or a vision model
- Caption/describe → embed the text
- Or use a multimodal embedder

Two takeaways

- A “one model understands everything” chatbot is really a router in front of several extractors — the model sees text after each type is converted.
- Semantic chunking assumes flowing text; for images/tables you chunk by structure (a table, a figure+caption) instead.

Design Decisions: Documents

The governing question: *what physical form is the text in, and how much of its value is in layout?* Get this wrong and every downstream step inherits the damage - 80% of bad RAG answers are decided here.

Design Decisions: Documents

1. **PDF, plain prose (papers, reports, specs)**

Choose: `pypdf`. **Because:** it's 5–10× faster and lighter than anything else, and plain prose has no structure to preserve. Don't reach for a heavier tool to solve a problem you don't have.

1. **PDF whose value is in TABLES (invoices, financials, datasheets)**

Choose: `pdfplumber` (or Camelot for ruled tables), serialize tables to Markdown. **Because:** `pypdf` flattens a table into a single word-salad line (`Q1 100 Q2 120 …`), destroying the row/column meaning your retriever needs. The speed cost buys back the numbers you came for.

Design Decisions: Documents

3. Scanned / image-only PDF (no text layer)

Choose: OCR - local `tesseract` (+ `pdf2image`), or cloud OCR / a vision model for hard scans.
Because: there is literally no text to extract; `pypdf` returns empty strings *silently*.
Always probe first - this is the trap that ships empty chunks to production:

```
def pdf_has_text_layer(path, sample=3):
        import pypdf
        r = pypdf.PdfReader(path)
        return any((p.extract_text() or "").strip() for p in r.pages[:sample])
# False -> route to OCR, never to pypdf.
```

The DPI / preprocessing why: OCR at `dpi=300` is the sweet spot (Tesseract was tuned near it); 72 dpi shreds small fonts, 600 dpi is ~4× slower for little gain. And cleaning the image (grayscale + binarization) usually helps accuracy *more* than adding pixels. The cheap win is a cleaner image, not a bigger one.

Design Decisions: Documents

4. Complex layout (multi-column, magazines, academic PDFs, forms)

Choose: a layout-aware parser - `Unstructured`, `docling`, `GROBID` (for papers), Azure Document Intelligence, or a VLM for the truly messy ones. **Because:** `pypdf` reads coordinates left-to-right and **scrambles multi-column reading order** - column 1 line 1, column 2 line 1, column 1 line 2… producing interleaved nonsense.

5. HTML

- **You control the site / stable layout** → `BeautifulSoup` with targeted selectors. **Because:** high precision — you keep exactly what you target.
- **Scraping the open web at scale** → `trafilatura` / readability. **Because:** every site differs; you can't hand-write selectors for all of them, so you want a heuristic that generalizes (high recall across unknown layouts).

The shared why: raw HTML is ~90% navigation, footers, cookie banners, and ads. Strip boilerplate *before* extracting text or every chunk is polluted with "Home | About | Contact."

Choosing the Right Tool for Document Parsing

Match your document type to the right tool for accurate, reliable extraction.

Document condition	Tool	Decided by (Why)
Plain text PDF	pypdf	Speed, no structure to keep
Table-heavy PDF	pdfplumber / Camelot	Tables die under pypdf
Scanned PDF	OCR (tesseract / VLM)	No text layer (probe first!)
Multi-column / forms	Unstructured / docling / Azure DI	Reading order + element tags
HTML, controlled	BeautifulSoup	Precision
HTML, open web	trafilatura	Robustness across layouts
DOCX	python-docx + headings	Free section metadata
Mixed at scale	router + Unstructured	One entry point

Key takeaway: Use the simplest tool that preserves the structure you need.
Garbage in → garbage out. Parsing quality directly impacts retrieval quality.



Structured data & tables: don't embed rows — query them

Relational data and big tables are answered by generating SQL, not by cramming rows into a vector store.

Why embedding rows fails

- Aggregations (SUM, AVG, COUNT, GROUP BY) can't be done by similarity search.
- “Top 5 customers by revenue in Q2” is a computation, not a nearest-neighbour lookup.
- Exact filters (date ranges, IDs) are what SQL is built for.

The pattern: Text-to-SQL (RAG over schema)

- Retrieve the schema/catalog (table + column descriptions), not the rows.
- LLM writes a SQL query for the user's question — queries are generated per question, not pre-listed.
- Run it; feed rows back to the LLM to phrase the answer.
- Hybrid apps route: prose → vector RAG, numbers/tables → SQL. A data-lake chatbot usually needs both.

A 200-page contract PDF: page 1 is a typed cover sheet, pages 2–200 are scans with no text layer. You run the pdf_has_text_layer() probe from earlier (sample=3) and route the whole file to pypdf if it returns True, else to OCR. How many pages end up with **no searchable text**?

1. 0 pages
2. 2 pages
3. 197 pages
4. 199 pages



A 200-page contract PDF: page 1 is a typed cover sheet, pages 2–200 are scans with no text layer. You run the pdf_has_text_layer() probe from earlier (sample=3) and route the whole file to pypdf if it returns True, else to OCR. How many pages end up with **no searchable text**?

1. 0 pages
2. 2 pages
3. 197 pages
4. **199 pages**

Fix: decide on OCR page by page

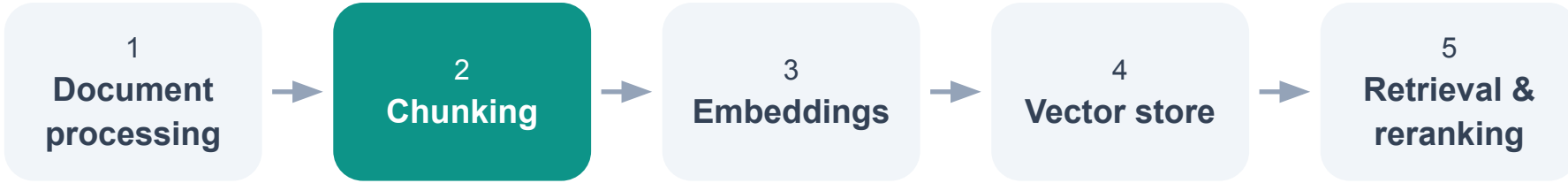
Check each page for a text layer and OCR only the empty ones (199 of 200 in Quiz 4).

```python
import pypdf, pytesseract
from pdf2image import convert_from_path

def extract_pages(path):
    """Text for every page; OCR only the pages with no text layer."""
    pages = []
    for i, page in enumerate(pypdf.PdfReader(path).pages, start=1):
        text = (page.extract_text() or "").strip()
        needs_ocr = not text                # scanned page: no text layer
        if needs_ocr:
            img = convert_from_path(path, 300, first_page=i, last_page=i)[0]
            text = pytesseract.image_to_string(img).strip()
        pages.append({"page": i, "text": text, "ocr": needs_ocr})
    return pages

pages = extract_pages("contract.pdf")
print(sum(p["ocr"] for p in pages), "of", len(pages), "pages needed OCR")
```

Design choices in a RAG system



Now: Chunking

Split documents into chunks that each hold one complete, self-contained answer.

Why chunk? Right-sizing for retrieval

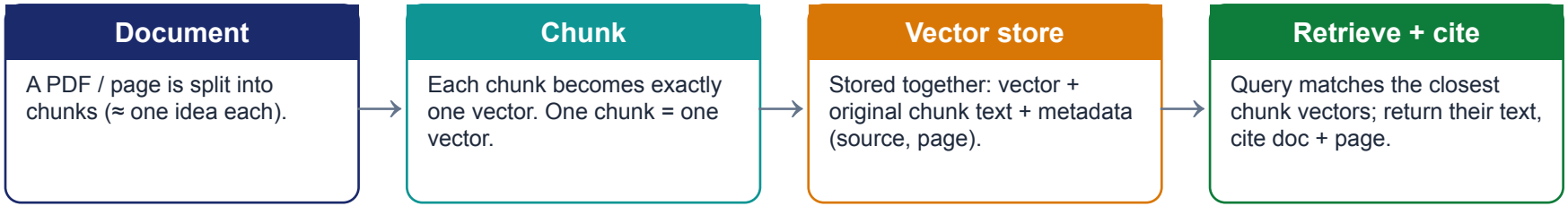
1. Documents are split into chunks, each embedded into one vector.
2. Similarity is computed chunk-by-chunk, so the chunk is the true unit of retrieval.
3. Embedding models also cap input length, forcing a split for long docs.

The goal

Each chunk should hold one self-contained, retrievable idea - small enough to be precise, large enough to stand alone.

What gets retrieved: chunk → vector → citation

RAG doesn't retrieve whole documents — it retrieves the chunks whose vectors are closest to the query, keeping the link back to the source.



Just the vector stored?

No — the chunk's text and metadata sit alongside it, so the LLM gets readable context, not a number.

Which doc / page?

Yes — metadata travels with each chunk, so answers can cite source and page. That's provenance.

Chunking (fixed-size · sliding window · semantic)

The governing question: *what is one self-contained "answer" in this corpus, and where are its natural seams?*

The eternal tension: too small loses context ("it rose 40%" - what rose?), too big dilutes the embedding and over-retrieves. Size should match how much text typically holds one complete answer in *your* data.

Four ways to chunk a document

Strategy	How it splits	Use it for	Watch out for
Fixed-size	Every N tokens	A baseline; uniform prose	Facts cut in half at boundaries
Sliding window	N tokens, ~10–20% overlap	Most RAG (the default)	20% overlap = 25% more vectors
Structure-aware	Headings, clauses, functions	Markdown, legal, API docs, code	Needs clean parsing first
Semantic	Where the meaning shifts	Topic-shifting documents	Embeds every sentence; cap the size

Start with a ~512-token sliding window and ~20% overlap; change it only when your retrieval tests say so.

1. Uniform prose, prototyping / a baseline

Choose: fixed-size, **token-based** (not character-based).
Because: simplest and fastest; you need a baseline to beat. Count tokens with *the embedding model's own tokenizer*, because the model's limit is in tokens - counting characters makes your budget a lie.

1. Prose where answers straddle cuts (most RAG)

Choose: sliding window with **~10–20% overlap**. **This is the sane default.**
Because: a fact spanning a chunk boundary gets split across two chunks and neither retrieves well; overlap guarantees it lands intact in at least one.

The dial: 0% = boundaries get cut; ~20% = catches them cheaply; 50%+ = near-duplicate chunks that double your store for marginal recall. 20% is the knee of the curve, not a magic number.

Chunking by tokens, with overlap

Count tokens with the embedding model's tokenizer, then slice the original text.

```python
from transformers import AutoTokenizer

# Count length in the EMBEDDING MODEL's tokens (all-MiniLM-L6-v2 reads <= 256)
tok = AutoTokenizer.from_pretrained("sentence-transformers/all-MiniLM-L6-v2")

def sliding_window(text, size=256, overlap=32):
    enc = tok(text, add_special_tokens=False, return_offsets_mapping=True)
    spans = enc["offset_mapping"]                # (start, end) chars of each token
    step = size - overlap                        # next window starts 224 tokens on
    chunks = []
    for s in range(0, max(len(spans) - overlap, 1), step):
        e = min(s + size, len(spans)) - 1
        chunks.append(text[spans[s][0]:spans[e][1]])   # slice the ORIGINAL text
    return chunks

chunks = sliding_window(open("hr_policy.txt", encoding="utf-8").read())
print(len(chunks), "chunks")                     # ceil((L - overlap) / step)
```

3. Strongly structured docs (Markdown, legal clauses, API docs, code)

Choose: structure-aware splitting - on headers, on clauses, on functions (AST for code).
Because: the document already tells you where the seams are. A chunk that *is* "§7.2 Termination" retrieves precisely for "termination clause" *and* carries provenance you can cite. A half-function or half-clause is useless to the LLM.

For code: split by function/class, never by token count
`RecursiveCharacterTextSplitter` has language-aware variants for this.

4. Heterogeneous, topic-shifting docs; quality > ingest cost

Choose: semantic chunking (cut where consecutive-sentence similarity drops).
Because: it puts boundaries at *meaning* shifts instead of arbitrary positions. The cost: you embed every sentence at ingest - expensive on huge corpora.
Always pair it with a hard `max_tokens` cap, or one long on-topic run blows the model limit.

Tuning chunking - and an overlap myth

Chunk on natural boundaries - sentences & paragraphs beat arbitrary character limits.

Common starting point: ~512 tokens, 10–20% (50–100 tok) overlap.

Recursive splitting (LangChain) respects structure then falls back to size.

Match chunk size to content: small for Q&A facts, larger for narrative/code.

Always evaluate retrieval & iterate - there is no universal best size.



Test, don't assume



Overlap is treated as a universal default — but a Jan-2026 arXiv study (SPLADE + 8B Mistral on Natural Questions) found **overlap gave no measurable benefit in their setup, only higher indexing cost.**



Semantic chunking can be **~14× slower** than token splitting (Chonkie benchmark).

→ *Spend that budget only when your metrics actually move.*

Choosing the Right Chunker for Your Data

General RAG Prose

Source-Grounded: Text synthesized directly from documents in a retrieved knowledge base. Information Dense: Prioritizes facts, data, and citations over stylistic flair.

Uniform Prose

Highly Predictable: Follows narrow vocabulary and repetitive sentence length. Statistically Homogeneous: Lacks the varied sentence structures, chaos, and distinct stylistic fingerprints of human writers.

Data condition	Chunker	Decided by
Uniform prose, baseline	Fixed-size (token)	Simplicity
General RAG prose	Sliding window ~20%	Boundary-spanning answers
Markdown / legal / code	Structure-aware	Natural seams + provenance
Mixed, quality-first	Semantic (+ cap)	Cut at meaning shifts
Chat / transcripts	By turn/speaker	The unit is an utterance
Dense reference QA	Small-to-big	Precision + context
Tables / CSV	Row / schema-aware	Don't split records

Measure it: chunking is the easiest choice to A/B — swap the chunker in the eval harness and re-run. The chunk-size knob alone often moves retrieval failure more than a model swap.

A team builds RAG for **PDFs with tables and multi-column layouts**. Accuracy is poor even though embedding model is strong. What is the most likely?

1. The cosine similarity is incorrect
2. The document parsing / chunking step is losing structure
3. The vector store is too small
4. The LLM context window is too large

Quiz 5



A team builds RAG for **PDFs with tables and multi-column layouts**. Accuracy is poor even though embedding model is strong. What is the most likely?

1. The cosine similarity is incorrect
2. **The document parsing / chunking step is losing structure**
3. The vector store is too small
4. The LLM context window is too large

Design choices in a RAG system

Five decisions, made in order. Each one caps the quality of the next.

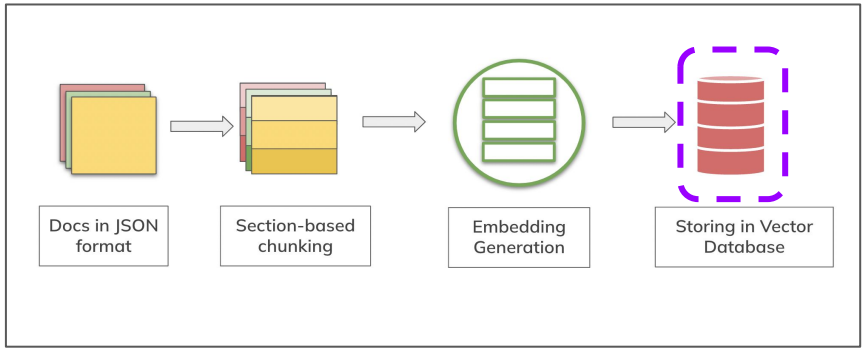


Now: Vector store

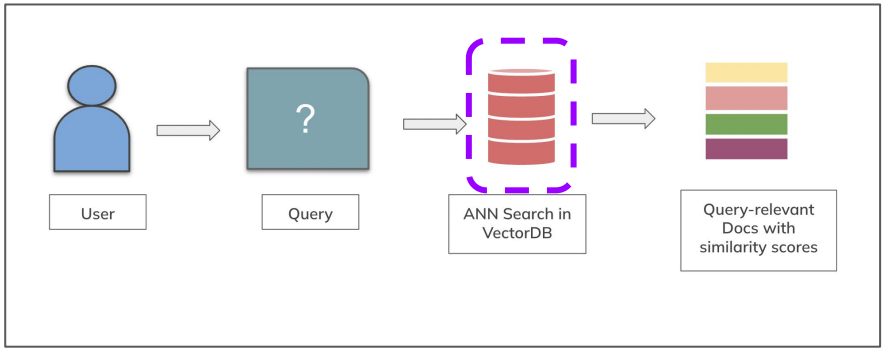
Store each vector with its text and metadata, and search millions of them fast.

RAG with Vector DB

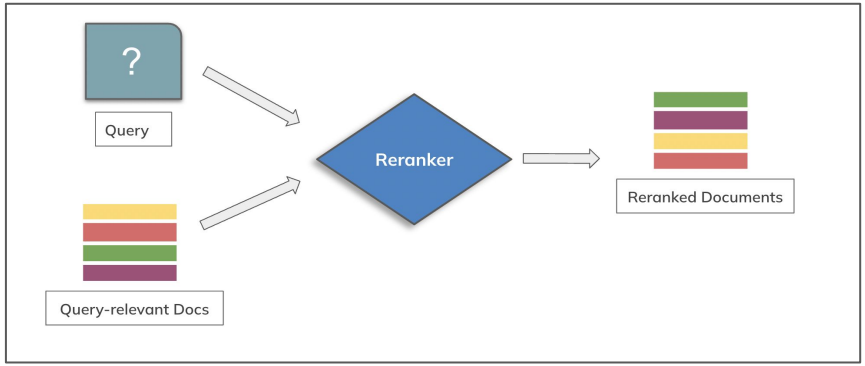
Pre-Retrieval



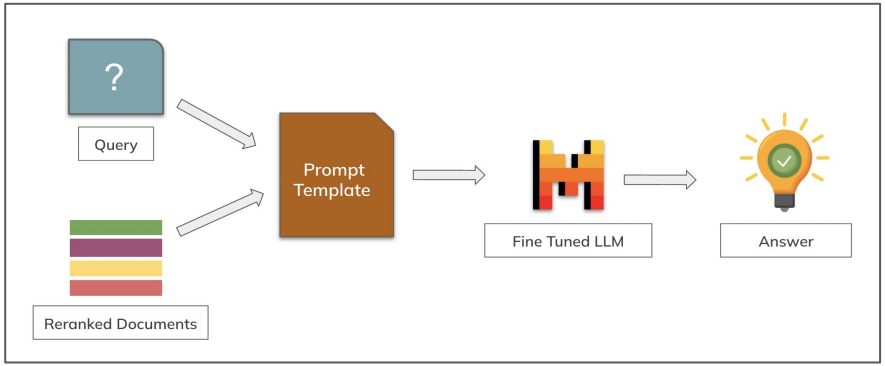
Retrieval



Post-Retrieval



Augmentation and Generation



https://ai.meta.com/blog/retrieval-augmented-generation-streamlining-the-creation-of-intelligent-natural-language-processing-models/

One row in the vector store: what actually gets saved

Each chunk becomes a single record — the vector for the math, the original text for the LLM, and metadata to filter & cite.

chunk_id: `doc_2024_hr_policy — p12 — c03`

vector

```
[ 0.0182, -0.0731,  0.1145,  0.0329, -0.0918, ... 763 more... ,  0.0447 ]
```

768 float32 numbers — this is what similarity search compares against

text

"Employees accrue 18 days of paid leave annually, credited monthly. Unused leave up to 30 days carries over into the next calendar year."

metadata

`source:` hr_policy_2024.pdf `page:` 12 `section:` "Leave" `date:` 2024-04-01 `lang:` en

Vector → find

Query is embedded the same way; nearest vectors = most similar meaning.

Text → answer

The matched chunk's text goes into the prompt — the LLM reads words, not numbers.

Metadata → filter & cite

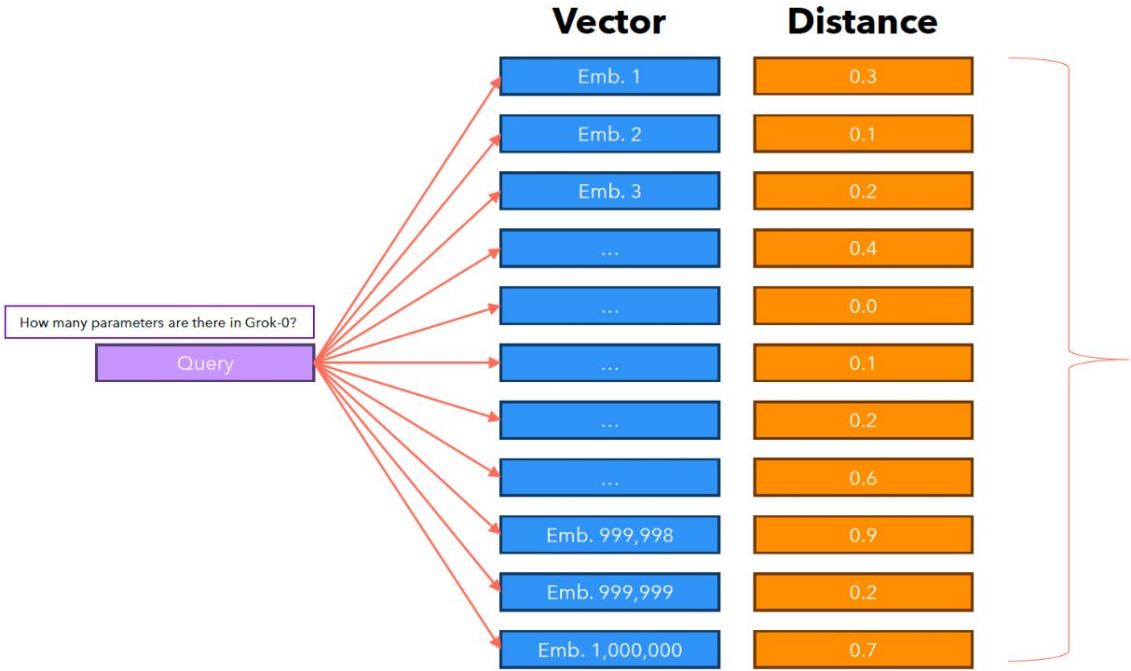
Pre-filter (source = HR) and cite "hr_policy_2024.pdf, p.12".

Why Efficient Vector DB is required for RAG?

1. A critical aspect that powers the capabilities of Retrieval-Augmented Generation models is the vector database that stores the embeddings for fast semantic search during the initial retrieval stage.
2. In order for RAG models to scale to immense corpora containing billions of text passages, efficiently indexing and querying vector representations is crucial.
3. Examples - Weaviate, Chroma, FAISS, Vespa, or Pinecone

KNN: A naive approach

Imagine we want to search for the query in our database: a simple way would be comparing the query with all the vectors, sorting them by distance, and keeping the top K.

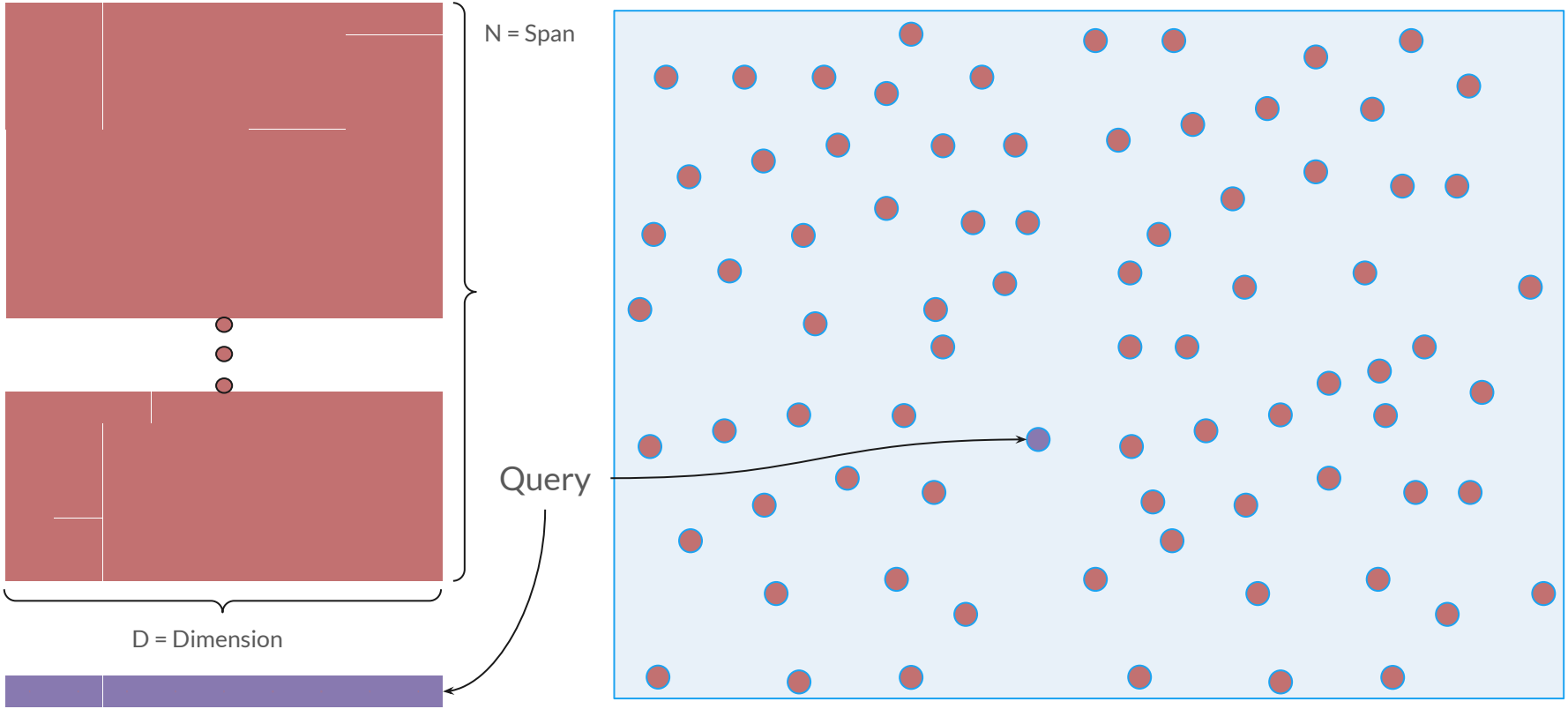




If there are N embedding vectors and each has D dimensions, the computational complexity is in the order of **O(N*D), too slow!**

Source: https://github.com/hkproj/retrieval-augmented-generation-notes

IVF: Inverted file for vector Database

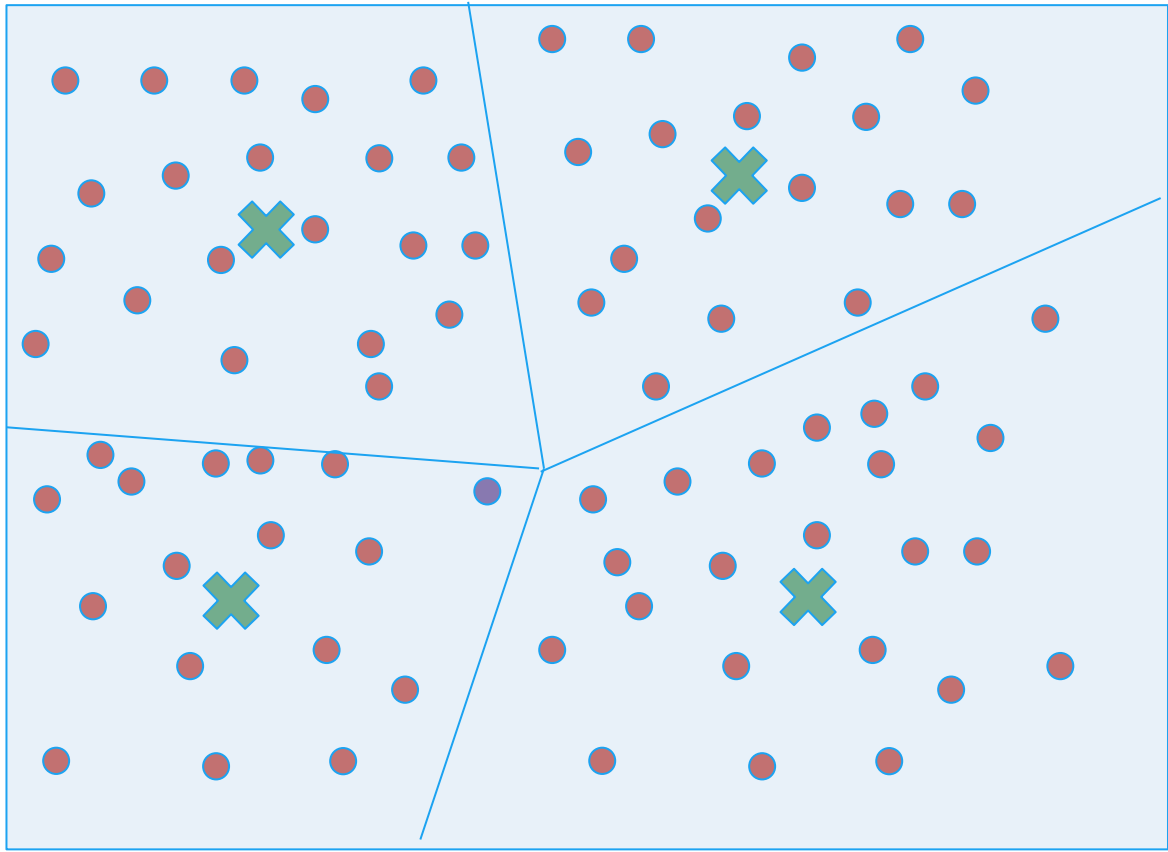


https://www.youtube.com/watch?v=BMYBwbkbVec&list=PLIUOU7oqGTLhlWpTz4NnuT3FekoulVlqc&index=9

IVF: Inverted file for vector Database

Phase 1: Indexing

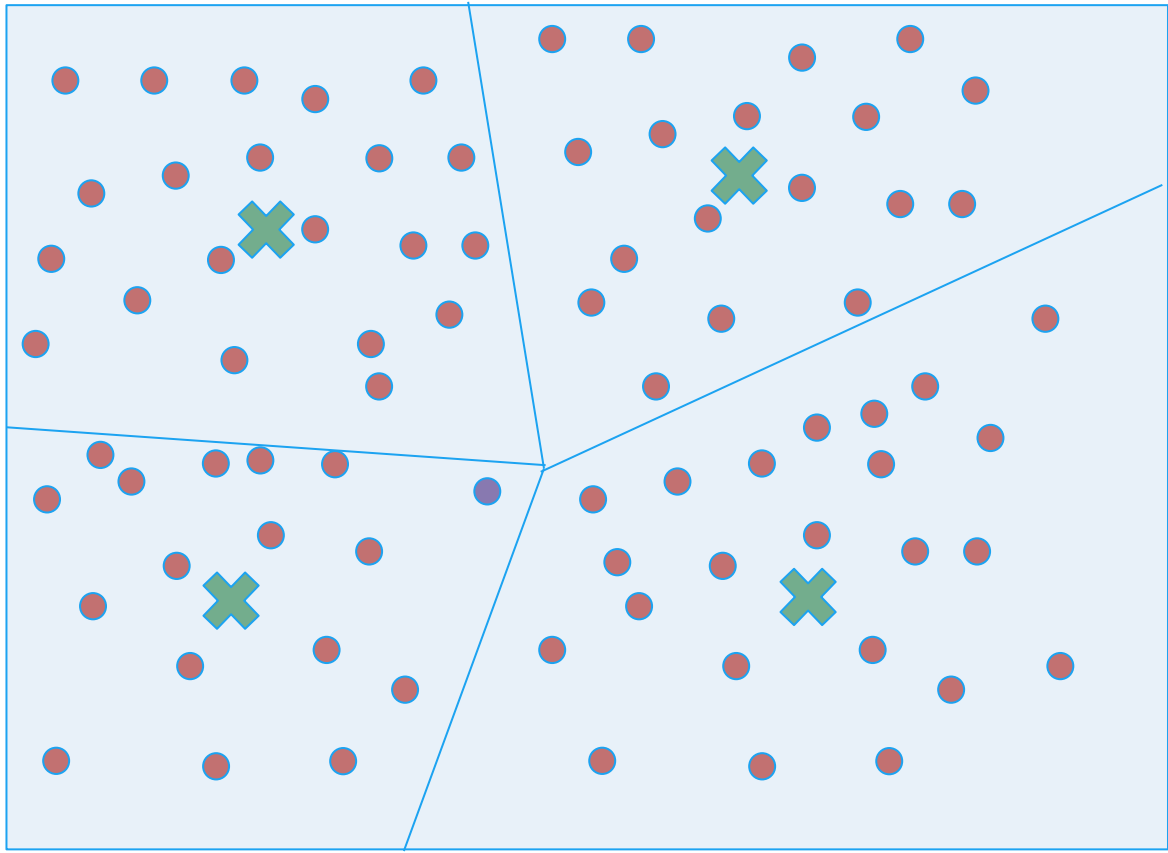
- **Train:** Run **K-Means** on the dataset to identify k representative centroids.
- **Partition:** Assign every vector to its nearest centroid (geometrically forming **Voronoi cells**).
- **Store:** Organize vectors into "buckets" (Inverted Lists) labeled by their centroid ID.



IVF: Inverted file for vector Database

Phase 2: Retrieval (The Search)

- **Coarse Search:** Compare the query **only** against the k centroids to find the closest match.
- **Fine Search:** Scan **only** the vectors inside that specific bucket (ignoring the rest)
- **nprobe** : #clusters (centroids) the algorithm evaluates during an ANN search (*=1 by default*)
- **Outcome:** Reduces search scope from O(N) to approximately O(N/k)



N = 5,000,000 vectors, nlist = √N buckets, nprobe = 2. Roughly how many vectors get scanned vs. brute force? Then: what does nprobe = 10 cost?

1. 2,10
2. 6708, 24596
3. 2048, 8192
4. 4472, 22360

Quiz 6



IVF: how much work do you actually save?

QUESTION

N = 5,000,000 vectors, nlist = √N buckets, nprobe = 2. Roughly how many vectors get scanned vs. brute force? Then: what does nprobe = 10 cost?

ANSWER

nlist = √5,000,000 ≈ 2,236 centroids, so each bucket holds ≈ 2,236 vectors. nprobe = 2 ⟶ 2,236 + 2 × 2,236 ≈ 6,708 comparisons vs. 5,000,000 — about a 745× reduction.

nprobe = 10 ⟶ 2,236 + 10 × 2,236 ≈ 24,596: still ~200× faster, with markedly higher recall.

Thank You

Instructor



Pawan Goyal

Teaching Assistants (IITKGP)



Pretam Ray



Kavin R V